



Save User Info with a Lex Chatbot

CH Arpita Chowdhury

Hello! Your Credit account balance, tucked away in bank of NextWork, is currently \$320.82.

and savings

For verification purposes, what is your date of birth?

10/01/1990

Thank you. The balance on your Savings account is \$179.8 dollars.



Introducing Today's Project!

What is Amazon Lex?

Amazon Lex is a service from Amazon Web Services that lets you build conversational chatbots using text and voice, and it is useful because it automatically understands user intent, manages conversation flow, and integrates with backend services like AWS Lambda to perform real actions without building everything from scratch.

How I used Amazon Lex in this project

In today's project, I used Amazon Lex to build a conversational chatbot that remembers user information across intents using context carryover, enabling smooth follow-up balance checks without repeatedly asking for the same details.

One thing I didn't expect in this project was...

One thing I didn't expect in this project was how powerful and flexible context carryover in Amazon Lex is, and how a small change to context expiry can significantly impact both the user experience and data privacy.



Arpita Chowdhury

NextWork Student

nextwork.org

This project took me...

This project took me 2 hours.



Context Tags

Context tags are labels used in Amazon Lex to store and share information between intents during a conversation. They allow the chatbot to remember details it has already collected (like a user's birthday) and reuse that information in other intents, instead of asking the user again.

There are two types of context tags: Input context tags – These control when an intent can be triggered, based on what has already happened in the conversation. Output context tags – These are set after an intent runs and are used to pass information forward to other intents. Together, they allow Amazon Lex to manage conversation flow and remember user information across intents.

I created a context tag called contextCheckBalance. This context tag was created in the CheckBalance intent. This tag stores information about the user having successfully checked their balance, allowing BankerBot to remember that this step was completed and carry related information forward during the same conversation.



Arpita Chowdhury

NextWork Student

nextwork.org

▼ **Contexts - optional** [Info](#)

Input contexts

Choose contexts



Output contexts

Choose contexts



contextCheckBalance



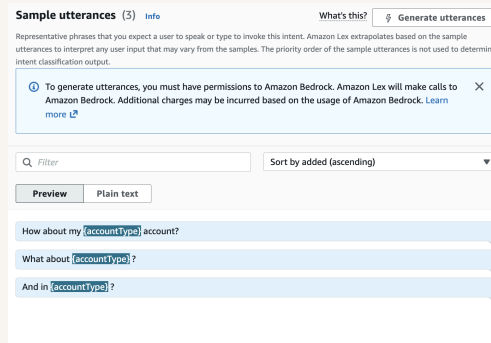
Expires in 5 turns or 90s



FollowUpCheckBalance

I created a new intent called FollowupCheckBalance. The purpose of this intent is to handle follow-up balance questions (like checking another account) by reusing previously collected information, without asking the user for their birthday again, ensuring a smoother and more natural conversation flow.

This intent is connected to the previous intent I made, CheckBalance, because it uses the input context set by CheckBalance to access saved user information (like the date of birth) and handle follow-up balance requests within the same conversation.





Input Context Tag

I created an input context, contextCheckBalance, that allows the FollowupCheckBalance intent to be triggered only after the CheckBalance intent has successfully run, ensuring the chatbot can reuse previously collected information like the user's date of birth for follow-up balance requests.

▼ **Default values - optional**

No default values
You haven't added any default values yet.

Provide a default value, #value for a context value, or [variable] for session variable.

#contextCheckBalance.dateOfBirth Add default value



The final result!

To see the context tags and follow-up intent in action, I tested BankerBot by first checking one account balance, then asking about another account in the same conversation and confirmed that the bot reused the saved date of birth instead of asking for it again.

If I had gone straight to trying to trigger FollowupCheckBalance without setting up any context, the intent wouldn't have worked correctly because Lex wouldn't know that the user had already completed CheckBalance or have access to the saved date of birth, causing the bot to either fail to trigger the intent or ask the user for their information again.



Arpita Chowdhury

NextWork Student

nextwork.org

Hello! Your Credit account balance, tucked away in bank of NextWork, is currently \$320.82.

and savings

For verification purposes, what is your date of birth?

10/01/1990

Thank you. The balance on your Savings account is \$179.8 dollars.



Managing context expiry

I updated my bot's context expiry to 1 turn or 5 seconds. What this means for my end users is that BankerBot will forget previously saved information very quickly, reducing how long sensitive details like a user's date of birth are remembered and improving privacy and security during conversations.

A long context expiry window would be helpful when users are likely to ask multiple follow-up questions in the same conversation and the chatbot needs to remember information to provide a smooth, uninterrupted experience. A shorter context expiry window would be helpful when handling sensitive information, where user privacy and security are more important and the chatbot should forget details as soon as possible.

The screenshot shows a modal dialog titled "Edit context" with a close button (X) in the top right corner. Inside the dialog, there is a label "Context tag name" above a text input field containing the value "contextCheckBalance". Below this, there is a label "Expires after" followed by two input fields: the first contains the number "1" and is followed by the text "turns, or", and the second contains the number "5" and is followed by the text "seconds". At the bottom right of the dialog are two buttons: "Cancel" and "Save".



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

